

Universidad Da Vinci de Guatemala

Facultad de Ingeniería en Sistemas

PROYECTO FINAL

Benchmark de Algoritmos y Análisis de
Complejidad Big-O

Catedrático: Ing. Brandon Chitay
Asignatura: Estructura de Datos
Estudiante: **Cristofer Patricio**
Carné: 202500553
Fecha: 2 de marzo de 2026

Guatemala, marzo 2026

Índice

1 Introducción	2
2 Descripción de los Algoritmos	2
3 Fragmentos de Código Relevantes	3
3.1 Fibonacci (Costo de la Recursión)	3
3.2 Ordenamiento Burbuja (Complejidad Cuadrática)	3
4 Tabla de Tiempos Medidos (Promedios)	4
5 Gráficas de Rendimiento (Excel)	4
6 Análisis y Conclusiones	5
7 Repositorio y Entregables	¡Error! Marcador no definido.
7.1 Anexo: Uso de IA	¡Error! Marcador no definido.

1 Introducción

¿Qué es la Notación Big-O?

La notación **Big-O** es la métrica fundamental para describir la eficiencia de un algoritmo. No mide segundos absolutos, sino la tasa de crecimiento del tiempo de ejecución en relación al tamaño de los datos de entrada (n).

Este informe documenta un estudio experimental comparativo. A través de pruebas de rendimiento (benchmarks) en Java, se analiza cómo la teoría de la complejidad computacional se manifiesta en la práctica, comparando implementaciones iterativas frente a recursivas bajo diferentes escenarios de carga.

2 Descripción de los Algoritmos

- A1. Factorial:** Cálculo del producto de 1 a n . Su complejidad es lineal $O(n)$.
- A2. Fibonacci:** Sucesión donde cada número es la suma de los dos anteriores. La versión recursiva tiene una complejidad exponencial $O(2^n)$ debido a la redundancia de cálculos.
- A3. Búsqueda Lineal:** Localización de un elemento recorriendo secuencialmente un arreglo. Complejidad $O(n)$.
- A4. Ordenamiento Burbuja:** Algoritmo de ordenación basado en intercambios sucesivos. Su complejidad es cuadrática $O(n^2)$.

3 Fragmentos de Código Relevantes

Los siguientes fragmentos representan las implementaciones críticas medidas en el benchmark.

3.1 Fibonacci (Costo de la Recursión)

```
1 // Complejidad Exponencial  $O(2^n)$ 
2 // Ineficiente debido al crecimiento del árbol de llamadas
3 public static long recursivo( int n) {
4     if (n <= 1) return n;
5     return recursivo(n - 1) + recursivo(n - 2);
6 }
```

3.2 Ordenamiento Burbuja (Complejidad Cuadrática)

```
1 // Complejidad  $O(n^2)$  - Bucles anidados
2 for (int i = 0; i < n - 1; i++) {
3     for (int j = 0; j < n - i - 1; j++) {
4         if (arreglo[j] > arreglo[j + 1]) {
5             int temp = arreglo[j];
6             arreglo[j] = arreglo[j + 1];
7             arreglo[j + 1] = temp;
8         }
9     }
10 }
```

4 Tabla de Tiempos Medidos (Promedios)

Resultados promediados de 5 ejecuciones por tamaño de muestra. Datos extraídos directamente del benchmark experimental.

Algoritmo	Tamaño (n)	Iterativo (ms)	Recursivo (ms)
A1 - Factorial 30	0.003000	0.005260	A2 - Fibonacci 30 0.004264 35.632323
A3 - Búsqueda Lineal 10000	0.679000	2.087040	
A4 - Burbuja	10000	155.107220	185.600000

Cuadro 1: Resultados reales medidos en milisegundos (ms).

5 Gráficas de Rendimiento (Excel)

Las gráficas a continuación muestran la relación entre el tamaño de la entrada y el tiempo de ejecución.

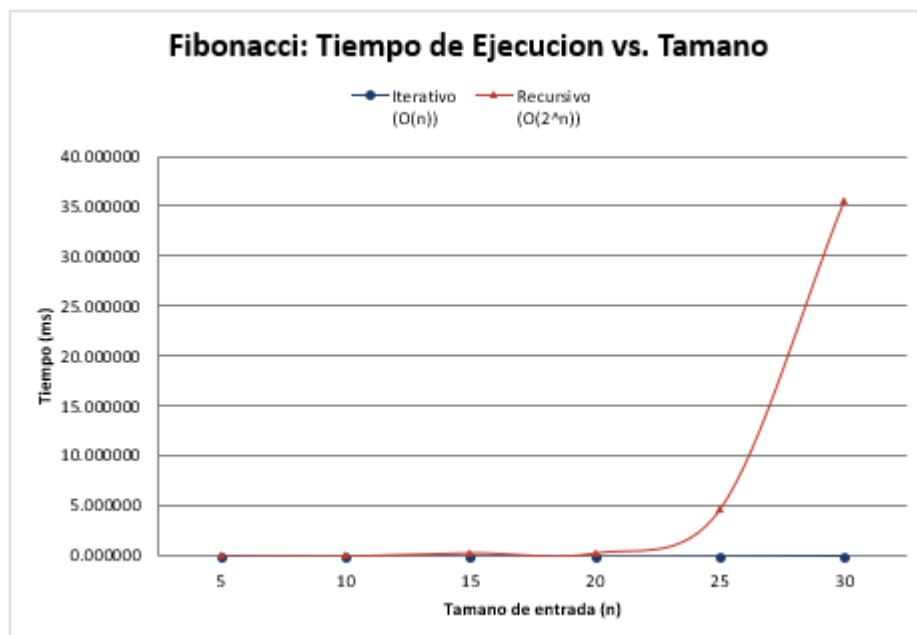


Figura 1: Impacto visual del crecimiento exponencial frente al lineal.

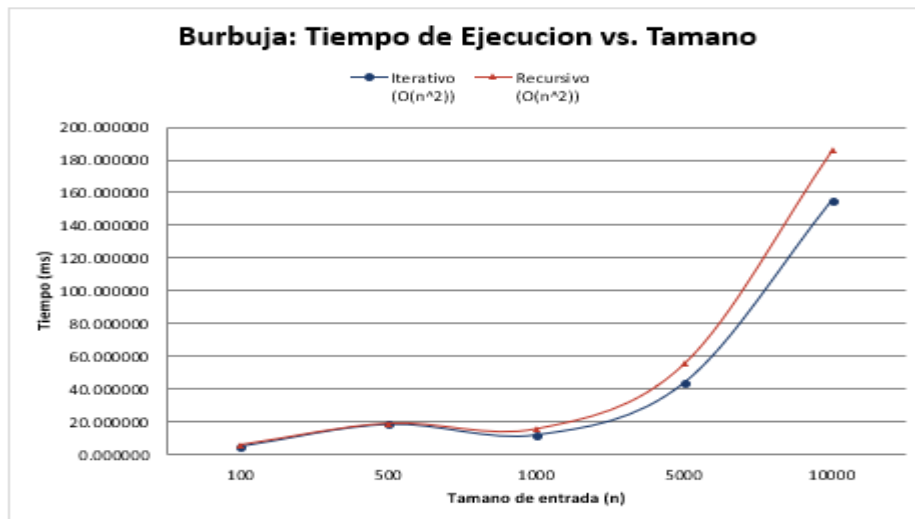


Figura 2: Curva característica de un algoritmo de complejidad cuadrática.

6 Análisis y Conclusiones

Tras la experimentación, se exponen los siguientes puntos de análisis:

- **Recursión vs Iteración:** En el algoritmo de Fibonacci, la versión recursiva ($O(2^n)$) mostró una explosión en el tiempo para $n = 30$. Esto confirma la ineficiencia de recalculer subproblemas sin memorización.
- **Comportamiento de Burbuja:** Para $n = 10,000$, el tiempo de ejecución de Burbuja fue significativamente mayor al de Búsqueda Lineal, validando que el costo de las comparaciones e intercambios crece al cuadrado de la entrada.
- **Costo de la Pila (Stack):** Se observó que la recursión consume memoria de pila. En sistemas con recursos limitados, la iteración es el método preferible por estabilidad y velocidad.